Documentação do sistema Easy Lab Correction beta

Alysson Milanez[1]
Augusto Macedo [1]
Demétrio Mestre [1]

[1]Departamento de Sistemas e Computação
Universidade Federal de Campina Grande (UFCG) – Campina Grande, PB – Brasil

{alyssonfilgueira, augustoqm, demetriogm}@gmail.com

Sumário

1 Introdução **2**

2 Motivação e Objetivo do Sistema **3**

2.1 Contexto 3
2.2 Atualmente 3
2.3 Estudo do conceito de Linha de Produto 3
2.4 Objetivo do Projeto 4

3 Stakeholders **5**

4 Requisitos Arquiteturais **6**

4.1 Requisitos Não-Funcionais 6
4.1.1 Requisitos de Produto 6
4.1.2 Requisitos Organizacionais 7
4.2 Requisitos Funcionais 9
4.2.1 Funcionalidades Selecionadas 9
4.3 Riscos Arquiteturais Críticos 9

5 Visão Lógica **10**

5.1 Questionário para escolha do Padrão Arquitetural 10
5.2 Decisões Chave para a Estrutura da Arquitetura 12

6 Bibliografia **13**

1. Introdução

O objetivo desse documento é descrever a decomposição do sistema Easy Lab Correction em subsistemas e estabelecer um framework de controle e comunicação entre estes, de forma que o mesmo possa prover um conjunto de serviços definidos pelo cliente. Assim, o projeto arquitetural facilitará a comunicação com os *stakeholders* (usuários, cliente, gerente de projeto, desenvolvedores e testadores), como também a análise do sistema com relação aos seus requisitos não funcionais e permitirá, principalmente, o reuso em larga escala da arquitetura, para linhas de produto por exemplo.

2. Motivação e Objetivo do Sistema

2.1. Contexto

Desde meados da década de 1960, o estudo de linguagens de programação é o primeiro alvo dos cursos de Ciência da Computação por todo o mundo, e naquela época já se pensava em como utilizar o próprio computador para avaliar os programas dos alunos. Muitas linhas de pesquisa foram desenvolvidas utilizando-se desse argumento: como avaliar o produto da criação de seres humanos?

Muitos anos passaram, novas descobertas, tecnologias e soluções foram desenvolvidas, no entanto, novos problemas surgiram, o número de estudantes aumentou, a quantidade de paradigmas e linguagens de programação criam polêmicas para a tarefa de escolher a mais apropriada para o ensino, a internet facilita a cópia de soluções prontas, o aumento da quantidade de tarefas dedicadas aos tutores e a delegação de atividades de correção aos assistentes (monitores) diminuiu a confiabilidade e aumentou os critérios de correção das atividades. Então, como avaliar atividades de programação atualmente, mantendo um padrão da correção e garantindo um bom nível de confiabilidade?

Esse problema foi identificado no Curso de Ciência de Computação da Universidade Federal de Campina Grande. A solução proposta é a criação de um ambiente de suporte a atividades de programação. O sistema *Easy Lab Correction* (ELC) tem como meta reunir as necessidades dos tutores, assistentes e alunos em um ambiente online de criação, correção das atividades, suporte (com ferramentas online) e submissão de soluções, discussão das notas e outras funcionalidades que propiciarão maior agilidade, padronização e segurança nas atividades de laboratório de programação.

2.2. Atualmente ...

O ELC encontra-se em uso na disciplina de Laboratório de Estrutura de Dados (LEDA) com o professor Adalberto Cajueiro (cliente do projeto). Esse estado de implantação constante do sistema (beta) juntamente com uma revisão bibliográfica sobre a automatização de correções de exercícios de programação permitiu a visualização de uma linha de produto de software nesse nicho. Para tanto um novo estudo arquitetural do sistema é necessário, dado que a versão atual não foi concebida com características de reusabilidade.

2.3. Estudo do conceito de Linha de Produto

Uma Linha de Produto de Software consiste em um conjunto de aplicações com uma arquitetura comum. Cada aplicação é especializada a partir de um subconjunto de funcionalidades centrais e genéricas comum.

Vários são os tipos de especialização que a família de aplicações pode ser desenvolvida [Sommervile 2010], a linha de produto ELC (beta) segue a *especialização funcional*, na qual versões de aplicações são criadas para clientes específicos com requisitos diferentes. Assim, apenas os componentes das funcionalidades do sistema devem adaptar-se ou novas funcionalidades devem ser adicionadas. Questões como mudança de plataforma ou de ambientes não são considerados.

Linhas de Produto de Software são projetadas para serem reconfiguradas, ou seja, devem ser genéricas e permitir facilmente sua especialização. Essa reconfiguração pode

acontecer em dois momentos, no momento da implantação ou no momento de projeto do novo membro da família de aplicações. O primeiro caso é amplamente utilizado em pacotes de software verticais projetados para aplicações específicas e com alto grau de similaridade, nesses casos a arquitetura deve ser amplamente reconfigurável não necessitando de acréscimos de funcionalidades no momento de implantação do software, o que requer ampla experiência da equipe de elicitação de requisitos e de desenvolvedores no nicho escolhido. A segunda forma de reuso ocorre em tempo de projeto, e partes do código fonte do núcleo do sistema pode ser modificado o que permite uma maior flexibilidade.

Para o projeto ELC foi escolhida a segunda alternativa devido a pouca experiência da equipe e, principalmente, o baixo número de funcionalidades atualmente desenvolvidas. Dessa forma, a equipe de desenvolvimento discutirá uma nova arquitetura que englobe não apenas as necessidades de um cliente, mas permita que uma série de requisitos funcionais e não funcionais sejam desenvolvidos ou adaptados.

Para adaptar uma linha de produto em uma única aplicação alguns passos são necessários:

1. Elicitar os requisites dos stakeholders;
2. Escolher o membro da família com maior similaridade;
3. Renegociar os requisitos;
4. Adaptar o sistema existente;
5. Entregar o novo membro da família ao cliente.

2.4. Objetivo do Projeto

Portanto, o projeto *Easy Lab Correction* na disciplina de Projeto I tem dois objetivos básicos: desenvolver uma arquitetura de linha de produto de software robusta com as características acima citadas e instanciar uma primeira aplicação para o cliente Adalberto na disciplina LEDA. Como trabalhos futuros (na disciplina de Projeto II) propõe-se buscar novos clientes com necessidades similares e assim expandir a família de aplicações.

3. Stakeholders

Os *stakeholders* envolvidos no Sistema são:

- Cliente e Professor: Adalberto Cajueiro, professor da disciplina de Laboratório de Estrutura de Dados;
- Monitores: alunos veteranos do curso selecionados para dar suporte à disciplina;
- Alunos: alunos da disciplina LEDA;
- Desenvolvedores, Testadores, Gerente de Projeto e Arquitetos: equipe de desenvolvimento do sistema, responsável por elicitar requisitos, planejar, desenvolver, acompanhar, testar e implantar o sistema.

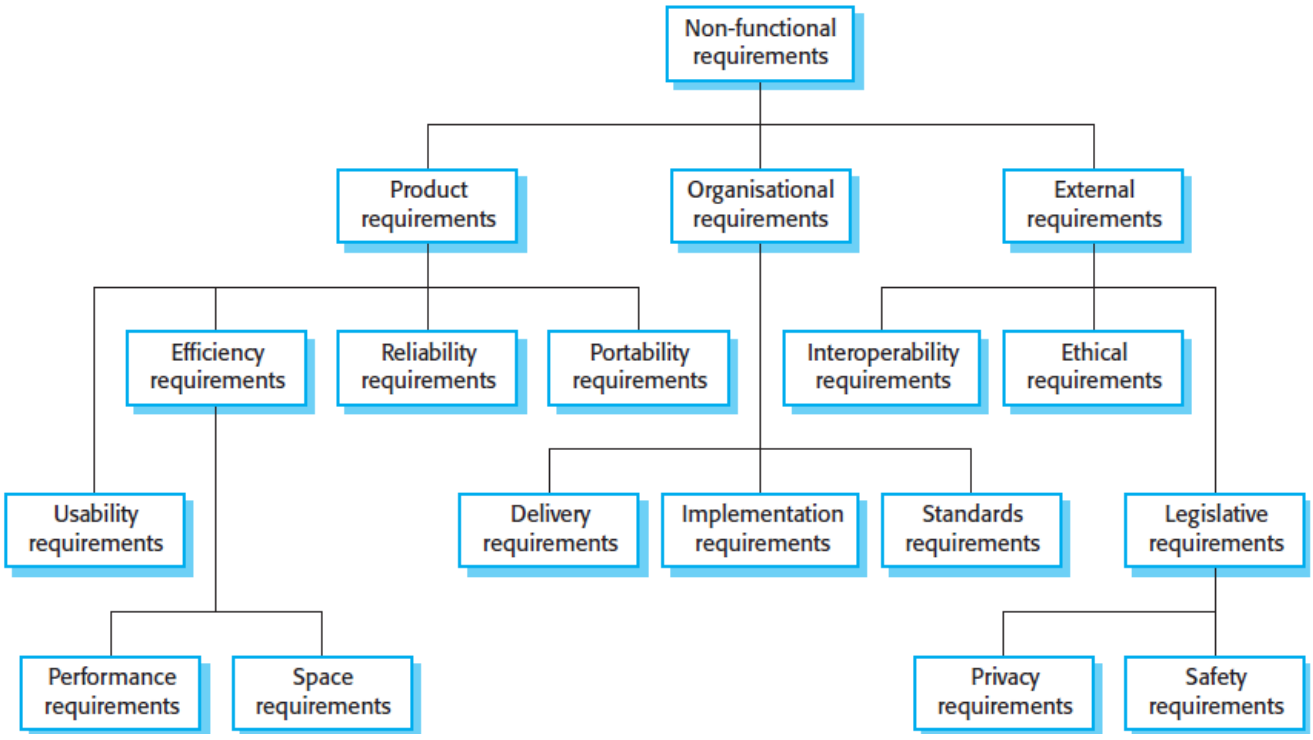


Figura 1. Tipos de requisitos não-funcionais [Sommerville 2010]

4. Requisitos Arquiteturais

4.1. Requisitos Não-Funcionais

Os requisitos não-funcionais (RNFs) da Linha de Produto ELC foram elencados e divididos em tipos de acordo com [SOMMERVILLE 2010], Figura 1. Alguns dos RNFs foram resultantes de acordos entre o cliente, professores da disciplina, que estabeleceram as datas das iterações e entregas, e os desenvolvedores, que já possuíam bom conhecimento de algumas linguagens e tecnologias.

4.1.1. Requisitos de Produto

A seguir são apresentados os requisitos de produto elencados para o sistema *Easy Lab Correction*.

Descrição	Métrica
Deve executar nos browsers web mais comuns	Número de browsers web com portabilidade
O sistema deve admitir entradas com codificação Unicode.	Número de instruções específicas de ambiente e linguagem

Figura 2. Requisitos de Portabilidade

Descrição	Métrica
Realizar Controle de Sessões	NA
Prover mecanismo de log de falhas	NA
Realizar backup dos dados constantemente	Tempo entre backups, ou seja, de perda de dados
Realizar testes no nível funcional e de unidade	80% de telas e código testado
Realizar análise estática e dinâmica da lógica do sistema	Número de erros detectados/corrigidos

Figura 3. Requisitos de Confiabilidade

Descrição	Métrica
Executar teste de Cargas	80 sessões simultâneas

Figura 4. Requisitos de Eficiência

4.1.2. Requisitos Organizacionais

A seguir são apresentados os requisitos organizacionais elencados para o sistema *Easy Lab Correction*.

Descrição	Métrica
Documentação Escrita (funcionalidades do sistema ELC – LEDA, especificação da Linha de Produto ELC e tutorial de uso)	Quantidade de telas documentadas
Não permitir que uma entrada para o sistema fique sem saída por mais que 5 segundos	Tempo máximo de resposta após falha de 5 seg

Figura 5. Requisitos de Usabilidade

Descrição	Métrica
Todas as datas das iterações estão previamente marcadas (ver http://www.dsc.ufcg.edu.br/~garcia/cursos/ProjetoI/)	NA
As funcionalidades de cada entrega podem ser renegociadas com o cliente durante o desenvolvimento do sistema	NA
Dar suporte ao sistema já implantado	NA
Permitir que o sistema seja facilmente instalado pelo cliente em outros ambientes	NA
Seguir a especificação dos requisitos funcionais descritos no checklist da linha de produto (próxima seção)	NA

Figura 6. Requisitos de Entrega

Descrição	Métrica
Documentação de Código.	Porcentagem de arquivos documentados
Refatoramento do Código	Número de arquivos refatorados
Internacionalização do código para inglês	Porcentagem de arquivos em inglês
Utilização de linguagem OO para desenvolvimento da lógica do ELC	NA
Utilização de banco de dados gratuito	NA
Utilização de servidor Web Gratuito	NA
Utilização de algum Servidor do DSC para hospedagem do sistema	NA
Utilização de sistema de Controle de Versões	NA

Figura 7. Requisitos de Implementação

Descrição	Métrica
Prover um serviço seguro a nível de aplicação (Camada de Aplicação, OSI)	NA
Aumentar a proteção contra códigos executáveis problemáticos, com bugs, ou até mesmo maliciosos	NA

Figura 8. Requisitos de Proteção

Descrição	Métrica
Garantir que cada usuário acesse apenas a sua conta	NA
Cada usuário de um determinado nível de permissão deve acessar apenas aquilo que está designado para seu nível de permissão	NA

Figura 9. Requisitos de Privacidade

4.2. Requisitos Funcionais

Por tratar-se de uma Linha de Produto, o ELC disponibiliza uma *checklist* com todas as funcionalidades passíveis de implementação e inserção em futuros membros da linha. Essa *checklist* deve ser disponibilizada ao cliente e deve ser preenchida como parte de um Requisito Não-Funcional de Entrega (descrito na seção anterior).

4.2.1. Funcionalidades Selecionadas

Em conversas com o cliente ficou acordado que o foco do projeto é o de manter o sistema ELC - LEDA, melhorar os RNFs, produtizá-lo e por fim, adicionar novas funcionalidades. Assim, os requisitos abaixo foram inicialmente selecionados.

- Criar tipos de roteiro
- Correção online das atividades, com adição de comentários ao código.

Para Projeto II almeja-se expandir o sistema para outros clientes e desenvolver novas funcionalidades da linha de produto, como por exemplo, a geração de casos de teste automática partindo de modelos e, a disponibilização de serviços para utilização de ferramentas online.

4.3. Riscos Arquiteturais Críticos

As decisões de projeto para o sistema ELC seguem um padrão de projetos amplamente utilizado pela comunidade, o qual reúne FLEX, JAVA e MYSQL. Essa combinação tem gerado bons frutos em diversos setores da engenharia de software, tais como produtividade, inovação, segurança, entre outros. No entanto, alguns riscos são de conhecimento geral.

A interface com o usuário, Flex + Action Script, apresenta dois riscos críticos, que podem ser contornados com a utilização de técnicas básicas.

O primeiro deles é a incompatibilidade entre os objetos em ActionScript com objetos Java. Caso não sejam tomadas às devidas precauções, a comunicação entre os dois pode gerar objetos ou atributos de objetos nulos, sem motivo e especificação prévia, o que compromete a confiabilidade do sistema. Uma solução para esse problema é o mapeamento de dados entre as duas linguagens, o que pode ser feito de duas formas: criação de BEANS (objetos semelhantes aos objetos Java, para realizar conversão de tipos) ou utilizar o padrão Command (Framework Cairngorm) que quebra os objetos em seus atributos e passa-os como parâmetros.

O segundo risco crítico está relacionado com a dependência do Adobe Flash Player para a exibição da interface gráfica. Constatou-se que plugins antigos do flash

player exibem interfaces incompletas e com falhas na execução, como é o caso das versões mais antigas para o sistema operacional Linux que não possuem mapeamentos de caracteres com acentuação, ou seja, caracteres acentuados nestas versões se transformam em caracteres indesejados. Para o correto funcionamento deste, é preciso manter o plugin atualizado nos terminais clientes.

A utilização de Servlets como provedores de serviços cria um risco adicional. Estes serviços utilizados para enviar e-mails, fazer download e upload de arquivos, utilizam os métodos doPost e doGet que podem ser acessados facilmente por atacantes, comprometendo assim a segurança do sistema. Para contornar esse problema, necessita-se unir a execução destes métodos a uma sessão do usuário, para que só sejam executados por usuários credenciados.

Outro risco crítico está na política de Backup do banco de dados. O administrador responsável pelo sistema deve sempre verificar se o backup dos dados está realmente sendo enviado para o local correto.

Por fim, o risco de estouro de memória ao realizar vários deployments no servidor Tomcat. É de amplo conhecimento da comunidade que o Tomcat não tem uma boa política de administração de deployments das aplicações. Com o Flex requer uma grande carga de deployment, por motivos desconhecidos provoca-se estouro de memória e ocasiona travamento temporário das aplicações hospedadas. Para contornar esse risco, por hora, deve-se evitar usar o Tomcat Manager e utilizar o deployment manual na pasta de aplicações.

5. Visão Lógica

5.1. Questionário para escolha do Padrão Arquitetural

Após analisar as condições do sistema implantado atualmente, as possíveis funcionalidades adicionais e os Requisitos Funcionais e Não-Funcionais do novo Sistema, foram respondidas algumas questões fundamentais para o projeto da arquitetura, baseadas em [SOMMERVILLE 2010].

- Existe uma arquitetura de aplicações genérica que pode agir como template para o sistema que está sendo projetado?
  Sim, o template de Transaction-Processing Systems, mais especificamente, de Sistemas de Informação. Esse tipo de sistema é caracterizado por permitir acesso controlado a uma grande base de informação, um banco de dados compartilhado por exemplo. O mesmo é modelado usando o padrão arquitetural de camadas (ou máquina abstrata, ver Figura 10).
- Como o sistema será distribuído através de processos?
  O ELC será distribuído entre os vários clientes e um servidor
- Qual ou quais estilos arquiteturais são apropriados para o sistema?
  O modelo de camadas unido com o cliente-servidor (ver Figura 11). O primeiro é responsável por organizar o acesso à base de dados. As múltiplas camadas modularizam o sistema, garantem portabilidade entre vários ambientes, por dividir deveres em vários níveis, e.g a camada inferior é a única a comunicar-se com o hardware. E, principalmente, flexibiliza a arquitetura horizontalmente, pois facilita o acréscimo de funcionalidades localizadamente na arquitetura, sem

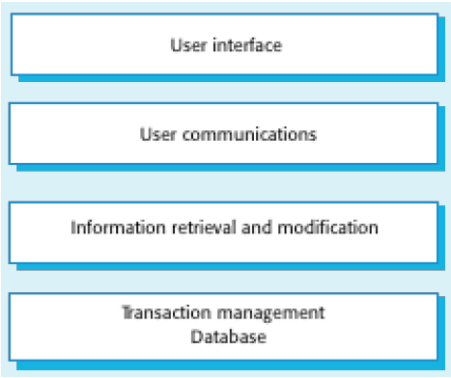


Figura 10. Modelo de Camadas ou Máquina Abstrata

preocupar-se com os níveis inferiores. O último permite que separemos o sistema em 3 níveis básicos: os clientes remotos, pela web; o servidor de processamento de aplicações, com os serviços e funcionalidades do sistema e; o servidor de administração de dados, o servidor de Banco de Dados. O ELC é uma linha de produto que oferece ambientes web de apoio ao ensino de programação. Portanto, a opção de arquitetura híbrida proposta permite ter a flexibilidade requisitada por uma linha de produto, a manipulação dos estados de um ambiente de programação e a disponibilidade da web.

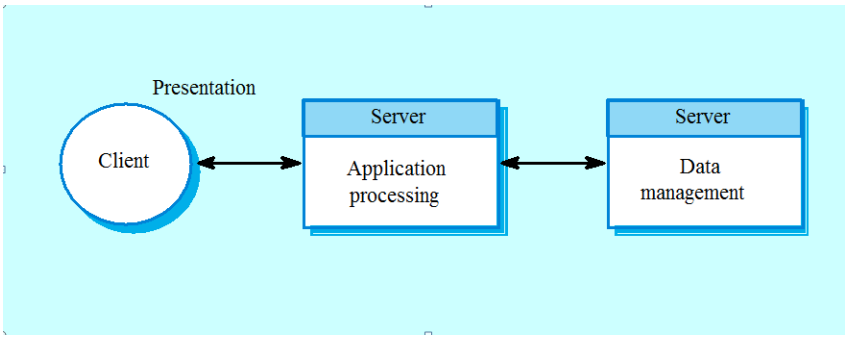


Figura 11. Arquitetura Cliente-Servidor de três camadas

- Qual será a abordagem fundamental usada para Estrutura o sistema?
  O modelo de camadas formará a base principal do sistema, no entanto, este terá alguns componentes adicionais da arquitetura cliente-servidor.
- Como as estruturas unitárias do sistema são decompostas em módulos?
  Iremos utilizar uma decomposição orientada a objetos, com uma modularização através de componentes.
- Qual estratégia será utilizada para controlar as operações das unidades do sistema?
  Duas estratégias serão utilizadas. A primeira será o controle centralizado com o modelo de call-return, que executará as requisições às camadas de comunicação, processamento e dados sequencialmente. E a segunda estratégia é o Broadcast model, utilizada para sistemas orientados a eventos, no caso, a GUI comunica-se com o restante do sistema através de eventos e as requisições de serviço ao servidor (modelo cliente-servidor) podem também ser considerados eventos.

- Alguma desvantagem pode ser prevista nesse modelo?
  Sim, as mudanças em camadas inferiores são custosas, as camadas diminuem também a eficiência e ao trabalhar com uma arquitetura híbrida o projeto do sistema e a criação de novos componentes deve ser cuidadosa para escolher a melhor forma de implementação.
- Como a arquitetura será avaliada?
  Pelo cliente inicialmente, em sala de aula através da avaliação de pares e pela professora, ao término.

5.2. Decisões Chave para a Estrutura da Arquitetura

	Decisão-Chave	Utilizar o Modelo Arquitetural de Camadas unido com o Modelo Cliente – Servidor
1	**Drivers**	Ver Seção: Questionário para Escolha do Padrão Arquitetural
	Descrição	---
	Vantagens	---
	Desvantagens	---
	Observações	---

	Decisão-Chave	Utilizar Flex com Plugin Flash Player
2	**Drivers**	RNF - Deve executar nos browsers web mais comuns
	Descrição	O sistema ELC deve ser capaz de funcionar nos principais *browsers web* utilizados atualmente.
	Vantagens	Os sistemas em Flex são portáveis para qualquer browser com o plugin flash player. Flex garantem uma certa segurança pois não permite acessos via URL. Possui uma interface amigável e elegante próxima dos anseios do usuário. Segue a vanguarda da computação nas nuvens pois disponibiliza uma plataforma para desenvolvimento de aplicações "desktop" através da rede (características como a atualização de páginas em tempo real, permitem essa comparação)
	Desvantagens	Versões antigas do plugin poderão introduzir "bugs" na GUI Os módulos SWF do Flex carregados para o computador cliente têm tamanhos na ordem de megabytes. O que tem levado várias equipes de desenvolvimento a desistir do uso da tecnologia.
	Observações	Atualmente os browsers e o próprio plugin atualizam-se automaticamente ao conectar-se na rede. O problema do tamanho dos SWF foi contornado com uma medida trivial, dividiu-se a aplicação em módulos, um por tela grosseiramente falando, e o usuário só carrega para sua máquina aquelas páginas que tem acesso.

3	**Decisão-Chave**	**Atualizar o servidor para que este aceite Unicode**
	Drivers	RNF - O sistema deve admitir entradas com codificação Unicode.
	Descrição	As entradas de campos nos formulários e os arquivos enviados devem manter sua codificação inicial
	Vantagens	Portabilidade
	Desvantagens	---
	Observações	Ao aceitar Unicode as outras codificações comuns estão englobadas

4	**Decisão-Chave**	**Utilizar C3PO para o controle de Sessões**
	Drivers	RNF - Realizar Controle de Sessões
	Descrição	O sistema deve controlar a quantidade de usuários que utilizam o sistema, juntamente com o servidor de aplicações
	Vantagens	Melhoria na robustez do sistema, ao conseguir lidar com um grande número de usuários simultaneamente
	Desvantagens	---
	Observações	---

5	**Decisão-Chave**	**Utilizar Log4J**
	Drivers	RNF - Prover mecanismo de log de falhas
	Descrição	Criação um mecanismo que dado a ocorrência de falhas no uso do sistema, seja gravado num arquivo o problema causador da falha.
	Vantagens	Possibilita descobrir o que está ocasionando falhas no sistema.
	Desvantagens	---
	Observações	---

6. Bibliografia

SOMMERVILLE, I. (2010) Software Engineering, Pearson Education, 10th Edition.

6	**Decisão-Chave**	**Realizar backup do MySQL e enviar para o email do Sistema semanalmente**
	Drivers	RNF - Realizar backup dos dados constantemente
	Descrição	Os dados do banco de dados devem sofrer backup constante e de forma automatizada, para garantia que possíveis danos ao servidor não comprometam por completo o uso do sistema. Os backups serão guardados no servidor e, logo após, enviados para um email. Essa foi a forma encontrada pelo cliente para levar os backups para uma máquina externa.
	Vantagens	Com os backups em mão é possível restaurar o banco de dados, garantindo que os dados armazenados não serão perdidos caso haja a ocorrência de problemas no servidor.
	Desvantagens	---
	Observações	Futuramente esse backup deverá também englobar o backup de arquivos do sistema. Esse intervalo de tempo pode ser modificado pelo cliente.

7	**Decisão-Chave**	**Testar todas as telas do sistema com o FlexMonkey e toda a Facade com o JUnit**
	Drivers	RNF - Realizar testes no nível funcional e de unidade
	Descrição	Devem ser realizados para o sistema ELC testes de GUI com FlexMonkey e de unidade com JUnit.
	Vantagens	FlexMonkey permite simular vários cenários de uso do usuário, como também a replicação de cenários com bugs declarados, a reexecução e a persistência dos casos de teste. O JUnit permitirá que executemos os cenários de uso do sistema através de sua Facade. Nesse nível de teste muitos dos casos tratados no código de GUI serão capturados pelo teste, obrigando o desenvolvedor a manter a modularização do código.

	Desvantagens	FlexMonkey requer um bom hardware, pois soma ao código compilado MXML do Flex seu código de teste. JUnit por ser um framework de teste de unidade não é muito indicado para teste de cenários. No entanto, ferramentas como o EasyAccept que automatizam esse processo mostraram-se pouco manuteníveis e com baixa produtividade.
	Observações	Nossa experiência prévia com JUnit diminui fortemente a curva de aprendizado, aumenta a produtividade, irá contribuir também para o aprendizado de uma tecnologia amplamente utilizada como funcionalidade do próprio ELC. Além de permitir a administração dos testes com o conceito de suites de teste.

8	**Decisão-Chave**	**Analisar a execução da Lógica do sistema (em Java) com o JavaPathFinder (realizar estudo prévio de viabilidade)**
	Drivers	RNF - Realizar análise estática e dinâmica da lógica do sistema
	Descrição	Deve ser utilizada uma ferramenta para realização de análise estática e dinâmica do sistema permitindo a verificação da sua execução (análise dinâmica) e da sua estrutura (análise estática).
	Vantagens	Verificar o comportamento do sistema em ambiente de execução. Previsão de deadlocks, starvation, e outros padrões indesejáveis de execução. Ferramenta que utiliza de modelos formais do código para deduzir possíveis problemas
	Desvantagens	---
	Observações	Ferramenta ainda não conhecida pela equipe. Para tal será realizado um estudo prévio de viabilidade